Introduction to black box testing

- Advantages and Disadvantages of black box testing
- Black Box Techniques: Boundary Value Analysis, Equivalence Class Partition, State Transition Technique, Cause Effective Graph, Decision Table, Use Case Testing
- Experienced Based Techniques: Error guessing, Exploratory testing

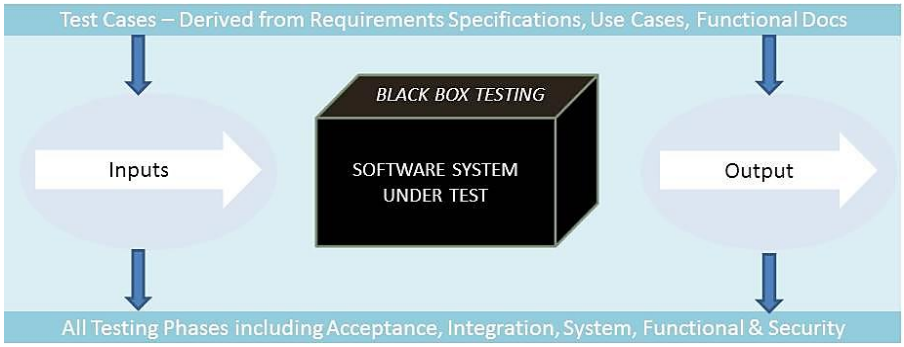
Level of Testing

- Functional Testing
  - Integration Testing and types - Top Down , Bottom Up , Non Incremental
  - System Testing
  - Acceptance Testing-Alpha and Beta
  - Smoke Testing
  - Regression Testing- Unit , Regional, Full
- Non-Functional Testing
  - Adhoc Testing
  - Performance Testing : Load Testing, Stress Testing, Volume Testing, Soak Testing
  - Recovery Testing

****

Introduction to black box testing

Black box testing is used to examine software functionality without checking the internal code of software. It just focuses on inputs and output of the software without worry about internal knowledge of the software program. The main purpose of Black Box is to check whether the software is working as per expected user specifications and requirements**.** The tester enters valid and invalid inputs to determine the expected outputs without having any deep programming knowledge. Black box testing also called as a behavioral testing or functional testing or specification based testing or closed box testing.



This testing technique can be applied on every level of testing such as unit, integration, system and acceptance testing.

This method tries to find following errors:

- Function errors
- Interfacing errors
- Database errors
- Performance errors
- Initialization errors

- **Black Box Testing - Steps**
  - Firstly requirements and specifications of the system are checked.
  - Tester chooses valid inputs (positive test case) and invalid inputs (negative test case) to check whether System under Test processes them correctly.
  - Tester verifies expected outputs for all those inputs.
  - Software tester writes test cases with the selected inputs.
  - The test cases are executed with compares the actual outputs with the expected outputs
  - Defects are fixed and re-tested.
- **Example:** An ATM machine used by general user, where perform the following functions at the ATM machine.

- Card Authorization
- Balance inquiry
- Cash Withdrawal
- Display Balance
- Generate receipt

You give input to the machine by just pressing/touching the buttons and the machine produces the desired output as dispatching the cash or displaying the balance etc. You don't know anything about the internal functioning of the ATM machine.

- If user gives invalid input i.e enters wrong ATM PIN then the system gives an error message
- If the correct PIN is entered, the system navigates to the next step.

Thus both the aspects valid and invalid input can be checked in Black Box testing

➢ Advantages of Black Box Testing

- Black box testing is more effective than white box testing.
- A testing team doesn't need knowledge of internal structure, logic implementation.
- No programming skills required for testing team.
- Test team and development team are independent of each other.
- BBT help to find the inconsistencies between implementation and specification
- Testing is done from the users' point of view, so it becomes easy to identify missing functions.
- Test cases can be designed as soon as the functional specifications are complete.

➢ Disadvantages of Black Box Testing

- It can be used for limited number of input and output combination.
- Test cases can be repetitive if the programmer has already covered in his unit test cases.
- Testing every possible path is difficult.
- The test cases are difficult to design.
- Time consuming
- It is difficult to identify all possible inputs in limited testing time. So writing test cases is slow and difficult

Black box or Specification Based testing Techniques

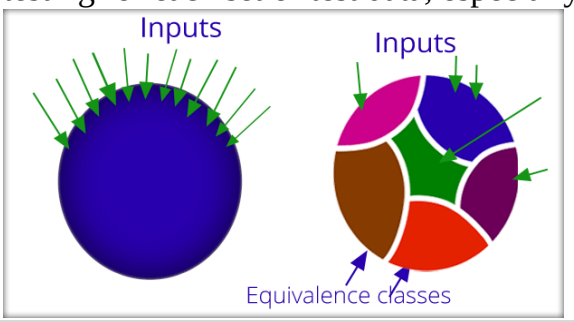
Test Design technique is a process to identify few test cases out of many possibility of identifying defects. It helps to achieve high test coverage. Specification based test techniques are based upon the test conditions and test cases derived from the SRS documents.

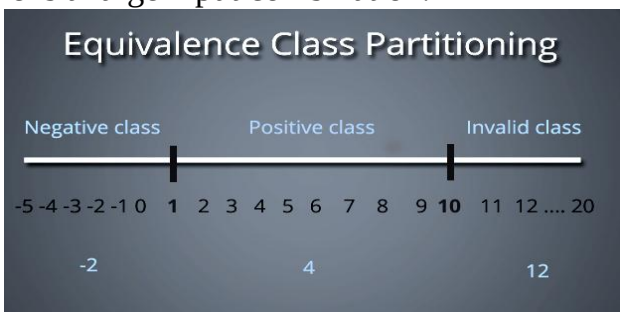
Types of Black box testing techniques

- ➢ Equivalence Partitioning
- ➢ Boundary Value Analysis
- ➢ Decision Table Testing
- ➢ State Transition Testing
- ➢ Use Case Testing

Equivalence Partitioning

This technique used for divide the test inputs into class (group) and test only one condition from each group. It is also known as Equivalence Class Partitioning (ECP). This approach is use to reduce huge amount of possible inputs to small. Due to time and budget, it is not possible to perform exhausting testing for each set of test data, especially when there is a large input combination.





In the equivalence partitioning input are divided based on the input values:

- ***Range:*** one valid equivalence class & two invalid equivalence classes.
- ***Specific set:*** one valid equivalence class & one invalid equivalence classes.
- ***Number:*** then we one valid equivalence class & two invalid equivalence classes.
- ***Boolean:*** one valid equivalence class & one invalid equivalence classes.

If an input from valid class is valid, other inputs are valid. If an input from invalid class is invalid, other inputs are invalid. It reduces lots of rework and also gives the good test coverage. We could save lots of time by reducing total number of test cases that must be developed. Equivalence partitioning is applicable at all levels of testing.

Example 1: Assume, we have to test a field which accepts Age 18 – 56

AGE Enter Age *Accepts value 18 to 56

EQUIVALENCE PARTITIONING		
Invalid	Valid	Invalid
<=17	18-56	>=57

- Valid Input: 18 – 56
- Invalid Input: less than or equal to 17 (<=17), greater than or equal to 57 (>=57)
- Valid Class: 18 – 56 = Pick any one input test data from 18 – 56
- Invalid Class 1: <=17 = Pick any one input test data less than or equal to 17
- Invalid Class 2: >=57 = Pick any one input test data greater than or equal to 57

We have one valid and two invalid conditions here.

Example 2: we have to test a filed which accepts a Mobile Number of ten digits.

MOBILE NUMBER Enter Mobile No. *Must be 10 digits

EQUIVALENCE PARTITIONING		
Invalid	Valid	Invalid
987654321	9876543210	98765432109

- Valid input: 10 digits
- Invalid Input: 9 digits, 11 digits
- Valid Class: Enter 10 digit mobile number = 9876543210
- Invalid Class: Enter mobile number which has less than 10 digits = 987654321
- Invalid Class: Enter mobile number which has more than 11 digits = 98765432109

We could not test every data like numeric, alphabets, alpha numeric, special characters and combination of alpha numeric and special characters. Here we can use Equivalence class technique to identify test cases. We can divide as classes or groups like

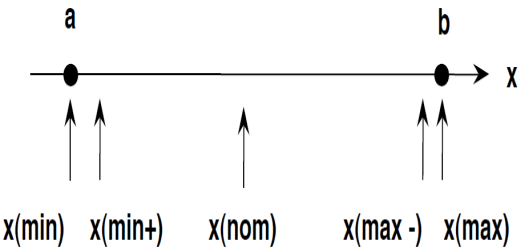
I. Only Numeric
II. Only Alphabets
III. Alpha Numeric
IV. Special Characters
V. Alpha numeric and Special Characters

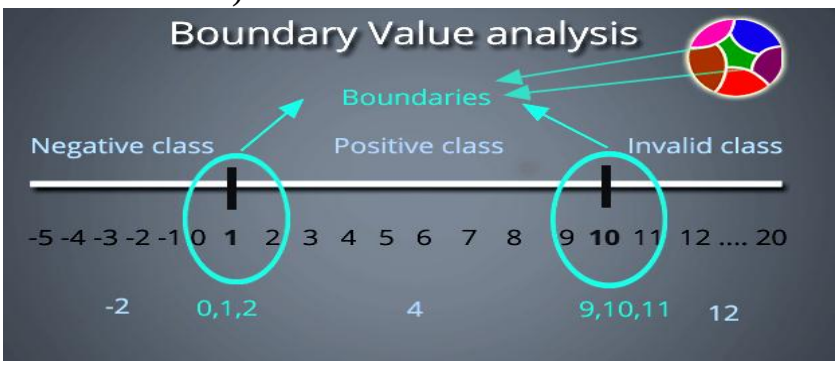
From each class we can select any test data randomly and use for testing.

Boundary value analysis(BVA)

Boundary value analysis is tests the boundaries between equivalence partitions as test conditions. BVA technique improves equivalence partitioning by focusing on the values at the ends of each partition or class. The basic idea in boundary value testing is to select input variable values at their:

1. Minimum
2. Just above the minimum(just outside boundaries)
3. A nominal value(typical values)
4. Just below the maximum((just inside boundaries)
5. Maximum





It is relatively easy to cover both valid and invalid boundary values. Its defect-finding capability is high. For each boundary, we test + and - 1 in the least significant digit of either side of the boundary. Every partition has its maximum and minimum values and these maximum and minimum values are the boundary values of a partition. Boundary value analysis can be applied at all test levels.

Example: Assume, we have to test a field which accepts Age 18 – 56

- Minimum boundary value is 18
- Maximum boundary value is 56
- Valid Inputs: 18,19,55,56
- Invalid Inputs: 17 and 57
- Test case 1: Enter the value 17 (18-1) = Invalid
- Test case 2: Enter the value 18 = Valid
- Test case 3: Enter the value 19 (18+1) = Valid
- Test case 4: Enter the value 55 (56-1) = Valid
- Test case 5: Enter the value 56 = Valid
- Test case 6: Enter the value 57 (56+1) =Invalid

AGE Enter Age *Accepts value 18 to 56

BOUNDARY VALUE ANALYSIS		
Invalid (min -1)	Valid (min, +min, -max, max)	Invalid (max +1)
17	18, 19, 55, 56	57

Decision table testing

Decision table testing technique used to test system actions for different input combinations. This is a systematic approach where a table showing combinations of inputs or causes with their associated outputs or actions (effects), which can be used to design test cases. This technique is called **'Cause-effect'** table. A decision table is useful when input and output data can be expressed as Boolean conditions like True, False, Valid, and Invalid. This table can ensure that a better test coverage.

Example: login page validation. Allow user to login only when both the 'User ID' and 'Password' are entered correct.

Conditions- Enter Valid User Name and Enter Valid Password.

Actions- Displaying home page or an error message if User ID or Password is wrong.

ID	CONDITIONS/ACTIONS	TEST CASE 1	TEST CASE 2	TEST CASE 3	TEST CASE 4
Condition 1	Valid User ID	T	T	F	F
Condition 2	Valid Password	T	F	T	F
Action 1	Home Page	Execute			
Action 2	Show a Message as 'Invalid User Credentials'		Execute	Execute	Execute

Rules →

Condition Stub	Condition Entry
Action Stub	Action Entries

State Transition testing

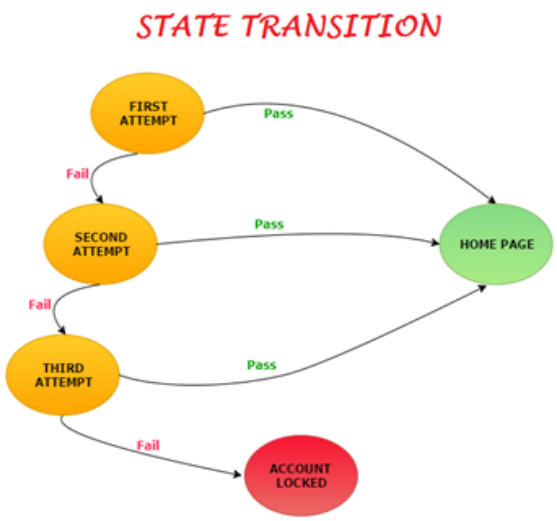
Tester analyzes the behavior of an application under test for different input conditions in a sequence. Tester provides both positive, negative inputs and Keep the record of system behavior. A system get a different output for the same input, depends on what happened before in the early state. **State Transition Testing Technique** is helpful where you need to **test different system transitions.**

The process of State transition testing to draw state transition diagram

- Determine start state, input, output and finish state
- Determine coverage level to be achieved
- Draw testing tree
- Define tests

Example: login page of an application which locks an user name after three wrong attempts of password.

STATE	LOGIN	CORRENT PASSWORD	INCORRECT PASSWORD
S1	First Attempt	S4	S2
S2	Second Attempt	S4	S3
S3	Third Attempt	S4	S5
S4	Home Page		
S5	Display a message as "Account Locked, please consult Administrator"		



First determine the states, input data and output data.

- Entering correct password, user will be redirected to the home page (state-S4).
- Entering incorrect correct password in the first attempt, a message will be displayed as try again and user will be redirected to state S2 for the second attempt.
- Entering incorrect correct password in the second attempt, a message will be displayed as try again and user will be redirected to state S3 for the third attempt.
- Entering incorrect correct password in the third attempt, user will be redirected to state S5 & a message will be displayed as "Account locked". Ask to Administrator".

Example 2: Withdrawal of money from ATM. 'User A' wants to withdraw 30,000 from ATM. Imagine he could take 10,000 per transaction and total balance available in the account is 25,000. In the first two attempts, he could withdraw money. Whereas in the third attempt, ATM shows a message as "Insufficient balance, contact Bank". Same Action but due to change in the state, he couldn't withdraw the money in the third transaction.

Error Guessing:

The tester can use his experience to guess the errors in application and its functionalities. Many defects can be found using error guessing where most of the developers usually make common mistakes. In Error guessing we don't follow any specific rules.

Few common mistakes that developers usually forget to handle:

- Divide by zero.
- File upload without attachment.
- File upload with less than or more than the limit size.
- Submitting a form without entering values.
- Entering invalid values such as entering alphabets in the numeric field.
- Testers can check if the developer handled null values in a field, text in a numeric field or numbers in a text-only field

Use case testing

Use case testing technique is helps to understand business conditions and flows. It is used to identify test cases that exercise the whole system on a transaction by transaction basis from start to finish.

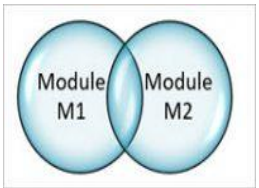
Use cases are a sequence of steps that describe the interactions between the actor and the system.

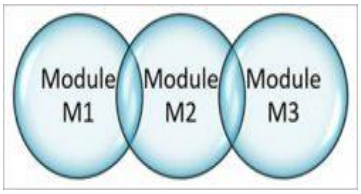
Example: In a use-case, an actor is represented by "A" and system by "S".

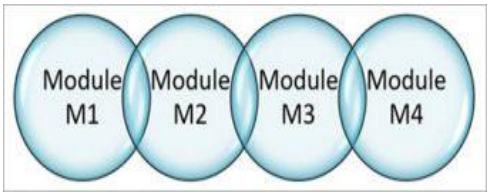
Main Success Scenario	Step	Description
A:Actor S:System	1	**A: Enter User Name & Password**
	2	**S: Validate Password**
	3	**S: Allow Account Access**
Extensions	2a	**Password not valid** S :Display Message and ask for re-try 4 times
	2b	**Password not valid 4 times** S :Close Application

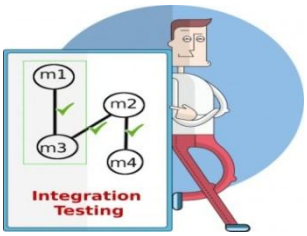
Integration Testing

Most of the Software applications are dividing into many modules or units and developed by different development teams. Each module tests individually in unit testing phase. Instead of integrating all modules at once & make a system. We are integrated every modules incrementally i.e. one by one all the modules or components are added (integrated) logically and make a complete system and performing testing on the system to check the original functional requirements are implemented correctly or not. Adding modules incrementally to ensure interface and interaction between modules and check the data flows between modules. Modules are added in a sequential manner. After integrating different modules then the integration testing is done.





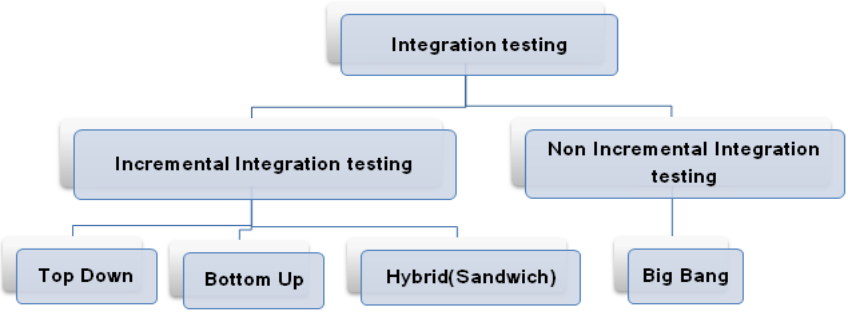




Integration Testing (In figure adding or integrate each module M1,M2,M3,M4 incrementally)
Integration testing will start after completion of unit testing. Integration Testing is the process of testing focuses on interactions between components.

➢ **Objective of Integration Testing or Why Integration testing requires:**

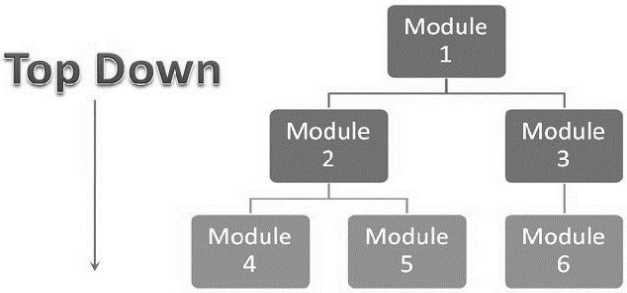
- Modules are developed by different developer whose understanding and programming logic may different from each other developer. Integration Testing ensure that different modules work together successfully after integration
- Interfaces of the software modules with the database or External Hardware could be wrong.
- Issues can be fixed in early phase without rework and in less cost
- Finding defects that occur when two or more modules integrated. If integration is not done properly, it will affect overall performance of software program.
- To check communication between integrated components is fine

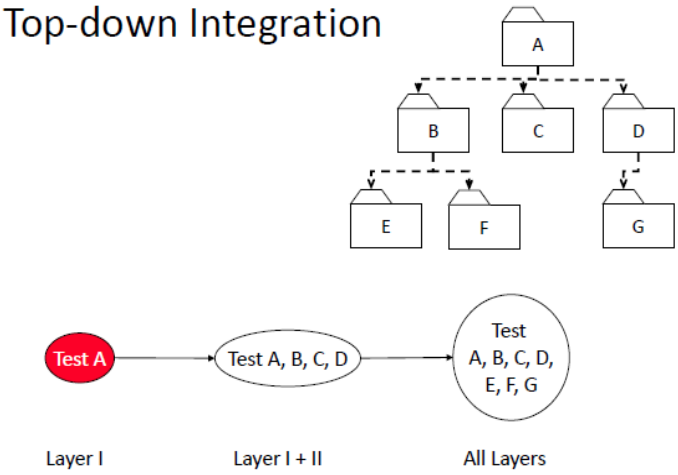


Example of Integration Testing:
A bike assembling where all components are integrated one by one or by big bang method and then tested if they are working in a required manner.
Stubs and drivers are pseudo code or dummy code used in Integration or component testing when one or more modules are not developed but are required to test some other module.

- **Stubs** are used in Top-down testing approach. Stubs help create the interface between lower level modules which are not available or developed.
- **Drivers** are used in Bottom-up testing approach. Drivers help create the interface between top level modules which are not developed.

Top Down Incremental Integration Testing
In top down integration testing, testing starts from top to bottom. Top level module is separately tested first and once they are tested successfully, then lower modules are integrated one by one. The process is repeated until all the modules are integrated and tested. The modules are added from the bottom.





Example

- Testing will start from module 1 then proceed to module 2, module 3 etc.
- In top down, Module 1 is tested separately remaining all modules integrated one by one and tested.
- If Module 2 is not developed but module 1 needed module 2 in order to test, so here use stubs.

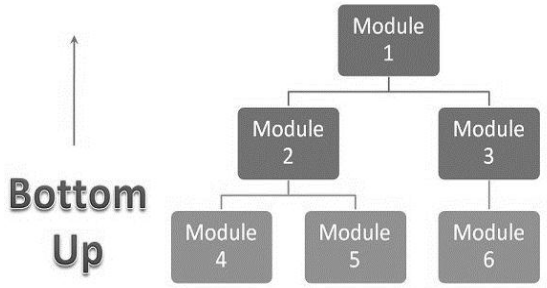
- So in place of module 2, stub is used. This stub will accept input from module 1 and returns the result. Stubs can be created by tester.

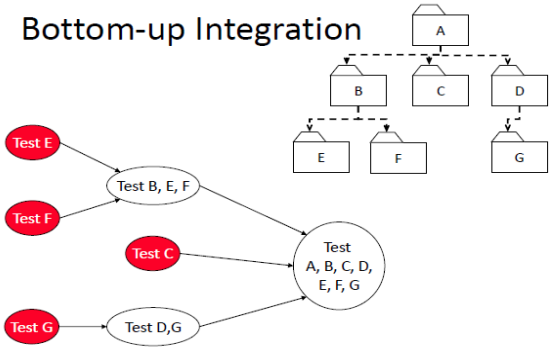
➢ **Advantages of Top down Approach:**
- Early introduction of design defects
- Main control points are tested early

➢ **Disadvantages of Top down Approach:**
- Needs many Stubs.
- A stub is not a perfect implementation of related Module
- Modules at lower level are tested poorly.
- Significant modules are tested late in cycle
- It is very challenging to write test conditions

Bottom up Incremental Integration Testing

In bottom up Approach testing, testing takes place from bottom to top, i.e. modules at bottom layer are integrated and tested first and then sequentially other modules are integrated as upwards to the higher level modules. When higher level modules are missing or not developed which affects on the integration process. In this situation to make the integration process possible then developer develops the temporary dummy program is known as 'DRIVER'. Unavailable or not developed modules are replaced by drivers.





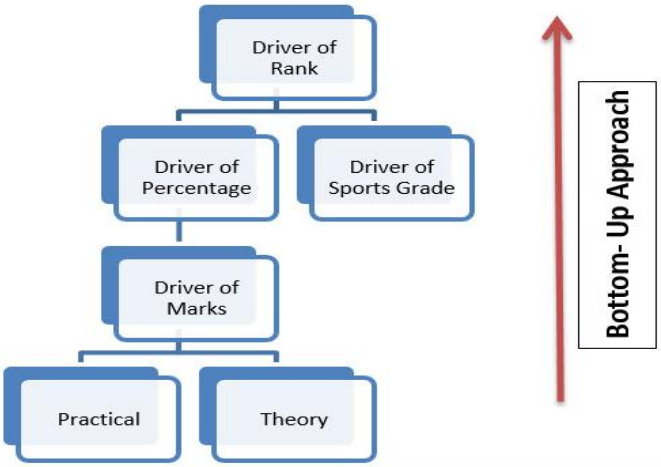
DRIVERS are the dummy programs which are used to call the functions of the lowest module in a case when the calling function does not exist.

Example
- Testing will start from module 4, module 5 and module 6 and this process will continue till modules are covered i.e. the process will come to halt once module 1 gets tested.

➢ **Advantage of Bottom-Up approach:**
- Testing starts from bottom level i.e. testing of critical modules or functionality at early stage.
- This helps in early finding of errors
- Interface defects are detected at early stage

➢ **Disadvantages of Bottom-Up approach:**
- Drivers are more difficult to write than stub
- Design defects are fixed in the later stage
- Driver is not a complete implementation of related Module.

Example: Modules Rank, Marks, Percentage and Sports Grade are not yet developed so they will be substituted with related Driver:



1: Unit testing of module Practical and Theory
2: Integration and testing of Modules Marks-Practical-theory
3: Integration and testing of Modules Percentage-Marks-Practical-Theory
4: Unit testing of Module Sports Grade
5: Integration and testing of Modules Rank-Sports Grade-Percentage-Marks-Practical-Theory

Top- Down Testing	Bottom-UP Testing
Top-Down testing conducted from main-module to sub module.	Bottom-Up testing conducted from sub module to main module.
If sub module is not developed then a temporary program module used called Stub	If main module is not developed a temporary program module used called Driver
Top -Down testing good if major error occurs to the top of the program.	Bottom-Up Testing good if major error occurs to the bottom of the program.
Test condition difficult to create.	Test condition easy to create.
Observation of test output is more difficult.	Observation of test output is easier.

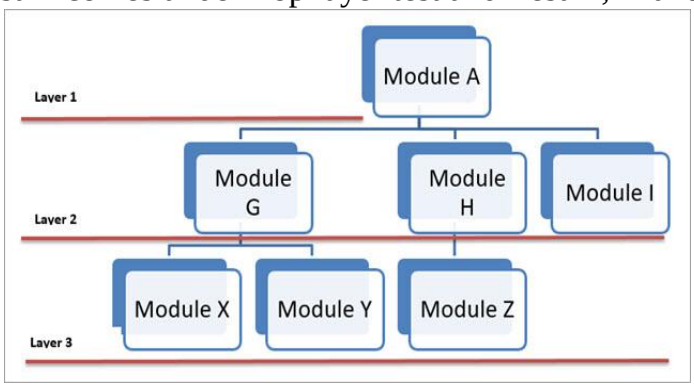
Hybrid (Sandwich) Testing

This approach is a hybrid of top-down and bottom-up methodology. Stub and drivers are used for incomplete or not developed modules.

- A middle layer is identified from which bottom-up and top-down testing are done. This middle layer is also known as target layer. i.e. select a layer which allows minimal use of Stubs and drivers
- Top-down testing starts from middle layer and moves downwards towards lower level modules. This layer below middle layer is known as Bottom layer
- Bottom-up testing also starts from middle layer and move up towards top layer modules. This layer above middle layer is known as Top layer
- With use of stubs and drivers, user interface and functions of lower level modules are tested respectively. At the end, only middle layer is left for the execution of the final test

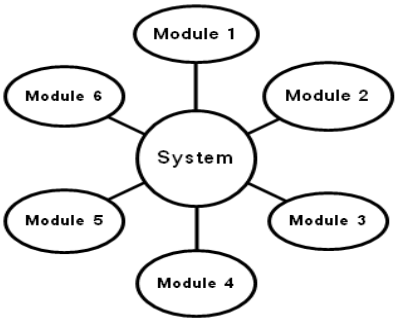
Testing Strategy:

1: Test A, X, Y, and Z individually – where Test A comes under Top layer test and Test X, Y and Z comes under Bottom layer tests
2: Test A, G, H and I
3: Test G, X, and Y
4: Test Hand Z
5: Test A, G, H, I, X, Y, and Z



Non Incremental Integration Testing

- It is also called as Big Bang Method.
- All the modules of the software are integrated simultaneously thus creating a big collection of modules and all the modules are tested together.
- In Big Bang integration testing all the modules is integrated without performing any integration testing and then it's executed to know whether all the integrated modules are working fine or not.
- This approach is generally executed by those developers who follow the 'Run it and see' approach.
- Because of integrating everything at one time if any failures occur then it become very difficult for the programmers to know the root cause of that failure.
- In case any bug arises then the developers has to remove the integrated modules in order to find the actual cause of the bug.



There are 6 modules named Module 1, Module 2, Module 3, Module 4, Module 5, and Module 6. These six modules are integrated simultaneously and all the six modules are tested all together. As these modules are not tested individually chances of occurring the failure may increase.

➢ **Advantages of big bang integration testing:**
- This type of testing requires very little or no planning.
- All the modules are completed before the integration testing starts.

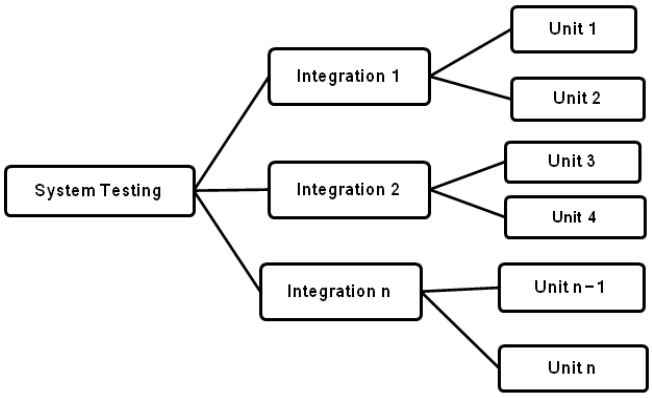
➢ **Disadvantages of big bang integration testing:**
- Since all the modules are tested together chances of failure increases.
- Difficult to find the root cause of the failure if failure occurs.
- Difficult to detach the modules if any error occurs.
- Time consuming process.

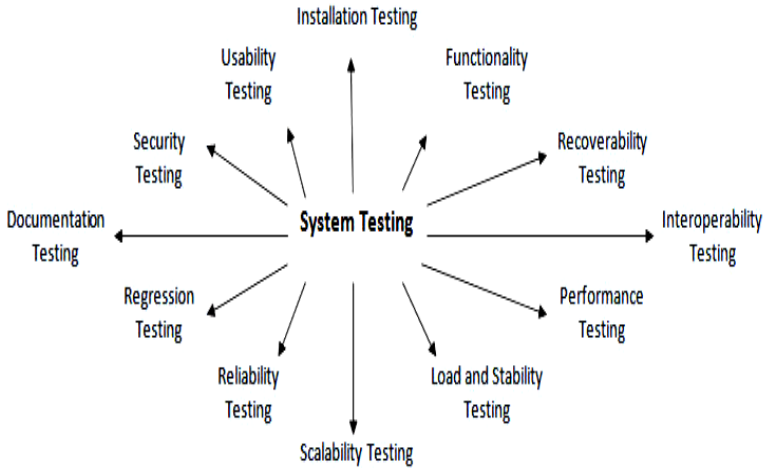
System Testing

- System testing is to check the functionalities, reliability, and other features of a complete and fully integrated software product based on the software requirements specification (SRS).
- The main focus of this testing is to evaluate Business/Functional/Non-functional/End-user requirements.
- System Testing is carried out by a testing team that is independent of the development team in order to measure the quality of the system balanced.

Objective of system testing:

- System testing removes risk by ensuring that it will function as per customer requirements.
- Testing check if system meets functional and non-functional requirement or not as per the customer's specifications.
- Validate that the system is complete and will work as expected.
- System testing aims to build confidence in the quality of the system as a whole.
- System Testing results are used by stakeholders to make release decisions.
- System testing may also follow legal or regulatory requirements or standards.
- Testers are focused on finding defects based on software application behavior, software design and expectation of end user.





System Testing is called a superset of all types of testing.

Example of personal computer (PC)

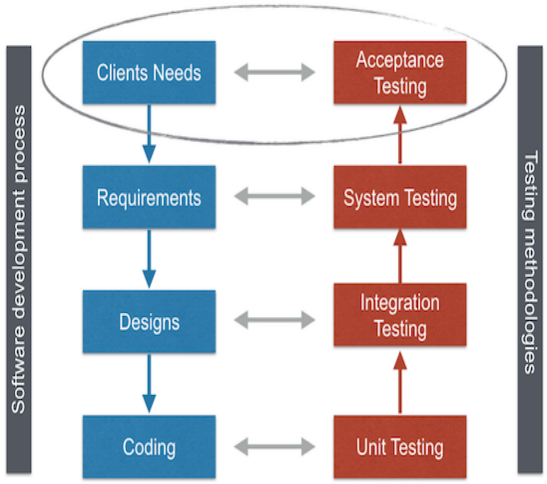
A PC has a monitor, motherboard, mouse, microprocessor, DVD player, RAM, hard disk, keyboard, etc. All of these parts are manufactured separately and individually tested before they could be assembled into a PC. Again different motherboard supports different microprocessor like 32 bit or 64 bit, different capacity RAM, hard disk, etc. Thus to carve a complete system, compatibility test is done after integration test. When complete system is assembled into PC, a system test is done to check all assembled part is working as expected. Once PC is ready it undergoes acceptance test done by the end user or the vendor who sells PC in the market.

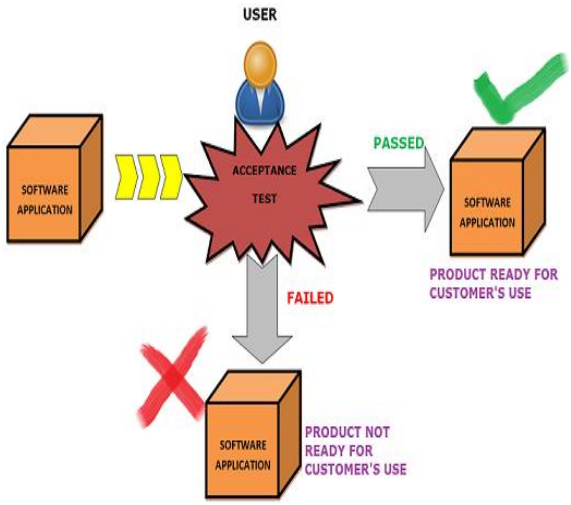
Acceptance Testing

- Acceptance testing is formal testing based on user requirements and function processing. It

determines whether the software is satisfy specified requirements and user requirements or not.

- Acceptance testing is the final level of software testing where the system is tested for acceptability.
- The user will test the system to find out whether the application meets their business needs. Once the acceptance test is passed then the application is released to production.
- Acceptance testing is performed by end user or customer with the purpose of to see if the product is ready for delivery.
- It is carried out after system testing and before final delivery to the client.
- The goal of acceptance testing is to establish confidence in the delivered system that it meets the end users requirements.
- The Acceptance testing is "black box" tests, means a users doesn't aware of internal structure of the code, they just gives input to the system & check whether systems respond with correct result.





- **Example:** when manufacturing a desktop computer: the monitor, keyboard, mouse are produced separately and all the parts are assembled together. An acceptance testing is performed so as to confirm that the desktop computer is ready to be made available to the end-user.
- **Example:** when manufacturing a ballpen, the cap, the body, the tail and snip, the ink cartridge and the ball pen are produced separately and unit tested separately. When two or more units are ready, they are assembled and Integration Testing is performed. When the complete pen is integrated, System Testing is performed. Once System Testing is complete, Acceptance Testing is performed so as to confirm that the ballpoint pen is ready to be made available to the end-users.

➢ **Before Start Acceptance Testing we need**

- Business Requirements must be available
- Application Code should be fully developed
- Unit Testing, Integration Testing & System Testing should be completed
- No defects in the System Integration Test Phase
- Only surface errors are acceptable before UAT
- Regression Testing should be completed with no major defects
- All the reported defects should be fixed and tested
- Traceability matrix for all testing should be completed
- UAT Environment must be ready

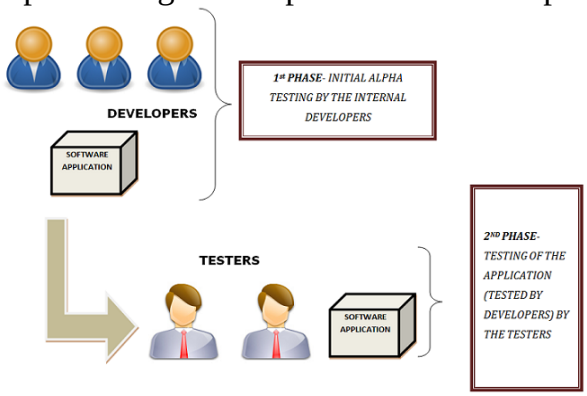
User Acceptance Testing (UAT):

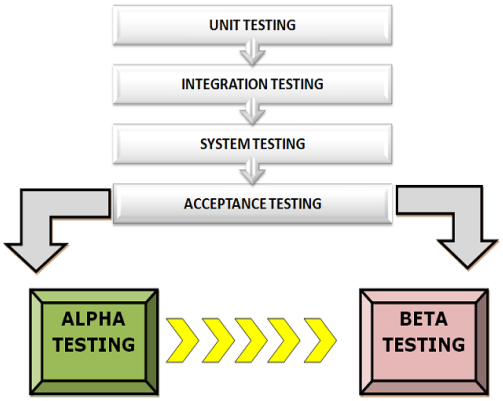
It is used to determine whether the product is working as per the requirements of the user correctly. Requirements as per the customers are picked up specifically for this purpose. This is also called as End-User Testing, Operational Acceptance Testing (OAT).

Alpha Testing

- Alpha testing is performed to identify all possible bugs before releasing the system.
- This is used in black box and white box techniques.

- Alpha testing is carried out in a lab environment and performed by the developers.
- Alpha testing is not open to market and public.





- Alpha testing is the final testing done by the company in two phases before its release.
  - ***First phase,*** it consists of developers who are responsible for carried out alpha testing with help of debugging software and locate bugs quickly.
  - ***In second phase,*** Testing team additional test application against user's needs and requirements in the real-environment.

➢ **Advantages of alpha testing:-**
- Provides view about software at an early stage.
- Provides early detection of errors.

➢ **Disadvantage of alpha testing:-**
- Depth testing cannot be done.

Beta Testing

- Beta testing is done at the client side that test whether the system is working as per requirements.
- Beta testing increases the quality and reduces failure rate of the product.
- Beta testing is open for the market and public.
- Only Black Box testing is involved in Beta testing.
- Beta version of the application will be released to limited number of users to get feedback regarding the quality.

➢ **Advantages of Beta Testing**:-
- Beta Testing improves the product quality via the customer feedback.
- Testers can fix all the issues and problems seen by the end user before release to public use.
- Users can install as well as test the application, and send their feedbacks to the developers.

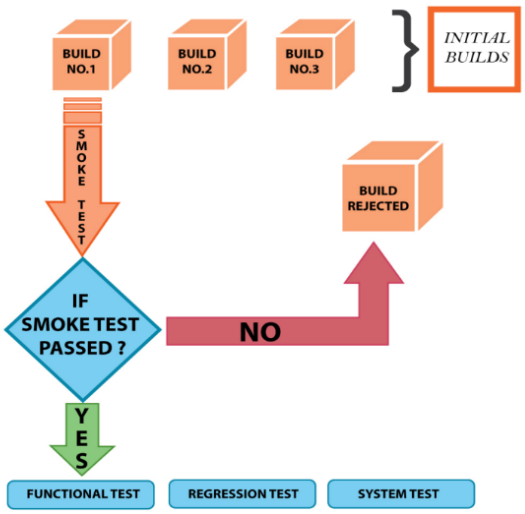
➢ **Disadvantages of Beta Testing:-**
- Testing is an issue because here the testing is done in the outside world of an organization.
- Finding the right beta tester is a difficult job.

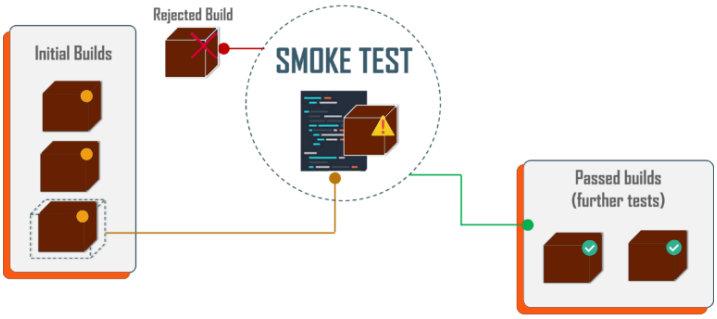
Alpha Testing	Beta Testing (Field Testing)
It is always performed by the developers at the software development site.	It is always performed by the customers at their own site.
Sometimes it is also performed by Independent Testing Team.	It is not performed by Independent Testing Team.
Alpha Testing is not open to the market and public	Beta Testing is always open to the market and public.
It is always performed in **Virtual Environment**.	It is performed in **Real Time Environment**.
It is always performed within the organization.	It is always performed outside the organization.
It comes under both White Box Testing and Black Box Testing.	It is only a kind of Black Box Testing.
Performed at the time when developers test the product and project to check whether it meets the user requirements or not.	Performed at the time when software product and project are marketed.
Alpha testing is not able to test the errors because the developer does not know the type of user.	In beta testing, the developer corrects the errors as users report the problems.

In alpha testing, developer modifies the codes before release the software without user feedback.	In beta testing, developer modifies the code after getting the feedback from user.

Smoke Testing

- Smoke testing is a process to check whether the installed build (application) is stable or not. If it confirm then QA team can continue with additional testing.
- QA team is verified to ensure the stability of the application. It is also called as "Build verification Testing".
- Smoke testing does not require to design test cases. There's need only to pick the required test cases from already designed test cases.
- Smoke Testing is performed on the software build to make sure that the most important and critical functionalities of the application are working fine or not.
- If any of the critical functionality of the application is not working, then build is testing immediately rejected by the tester.
- After rejection of (build) application, a developer fixes the issues and deploys the application for performing again Smoke testing.
- This type of testing is applicable in the Integration Testing, System Testing and Acceptance Testing levels.
- Most of the defects are identified at initial stage. i.e. it ensures the correctness of the system in initial stages.
- QA team can find defects to the application functionality that may have faced by the new code.
- Smoke Testing is done on the software build before any detailed Functional or Regression Testing is started
- Smoke testing saves the time and efforts by rejecting build before getting detailed Functional or Regression Testing

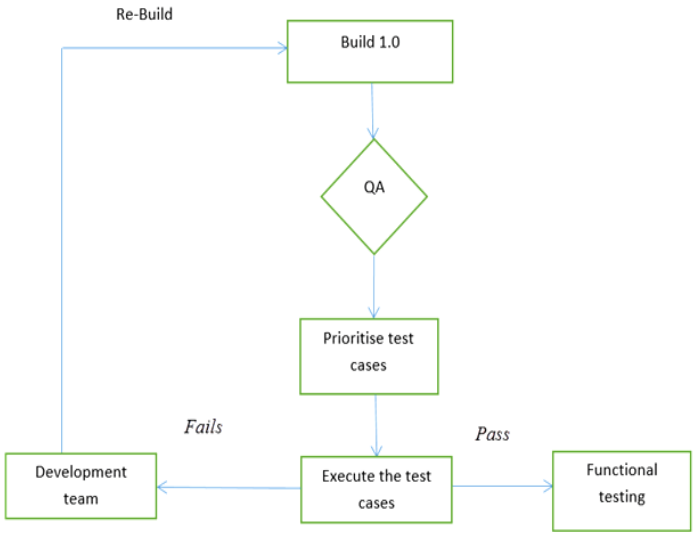




- Smoke Testing normally takes a lots of time and should be done for every new build, every new release even. Once the product is stable, automating the smoke tests.
- Whenever Smoke Testing is done if the new functionalities of software are developed and integrated with existing build that is deployed in QA environment. It ensures that all critical functionalities are working correctly or not.
- **Example of Smoke Testing is FACEBOOK with set of Smoke Test cases:**
  - User is able to Load the application
  - Application is responsive enough
  - User is able to login
  - User can create, edit, delete his profile
  - User is able to view other profiles, groups, pages etc.
  - User is able to create groups, pages etc.

- Chat functionality is working
- User is able to view the notification
- User is able to upload the pictures
- User is able to share updates

- **Smoke testing cycle**
  Once a build is deployed to QA, the basic cycle followed is that if the smoke test passes, the build is accepted by the QA team for further testing but if it fails, then the build is rejected until the reported issues are fixed.



➢ **Advantages of smoke testing**

1. It helps in quickly identifying issues in the crucial functionality.
2. It saves the testers time by avoiding testing an unstable or wrong build
3. It helps to find integration issues faster and minimizes it.
4. Saves test effort and time
5. To find issues and defects will be identified in early phase of testing.
6. Improves the quality of the system
7. Reduces the risk
8. Security, privacy policy, performance, etc. can also be tested
9. Detect critical errors and correction of errors is easy.
10. It runs quickly
11. It provides confidence to the tester to proceed with testing

Example: Suppose, we are using an eCommerce site, and the core working of this site should be login, specific search, add an item into the cart, add an item into the favorite, payment options, etc. Here we are testing function to place an order. After testing, the tester has to be sure and confident about the functioning of the function of the application.

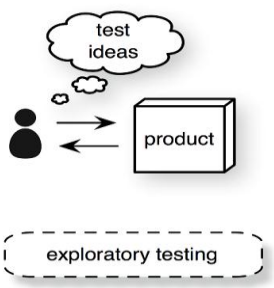
Functions are:

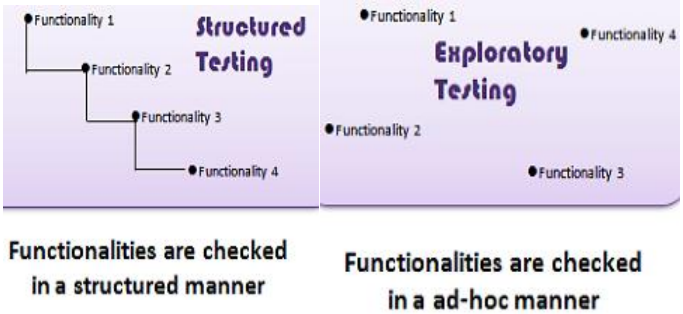
- Click on item
- Description page should be open.
- Click on Add to Cart
- The cart should be open
- Click on Buy Now
- Payment options should be displayed. Choose one of them.
- Order placed

Exploratory Testing

- This is formal testing process where tester don't have any test cases or test planning to test the application i.e. Testers navigate through the software without any specific goal.
- Testers (QA) are involved in maximum test execution and minimum planning.
- Tester opens the application and just starts to identify defects.

- The testing quality is also dependent upon the tester's knowledge about the product to design test cases and the different test methods for finding defects.
- Exploratory testing learns about behavior of the software, exploring about the software's, what it doesn't do and what it does, what works and what does not work.
- We can perform the test design and activities parallel without formal document, the test cases, test conditions or test scripts.
- This approach is used when poor specifications and where time is limited.



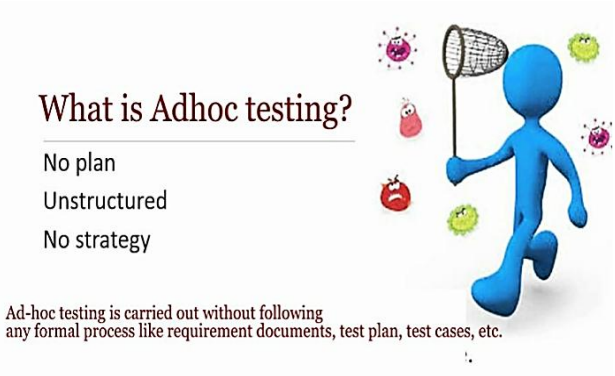


Example: if you observe a monkey in a zoo, you will see it jumps from one place to another place without following a pattern or any reason. In a similar fashion, in ET, the tester does not follows any pattern and just (navigates) jumps from one part of the application to another to see how it performs.

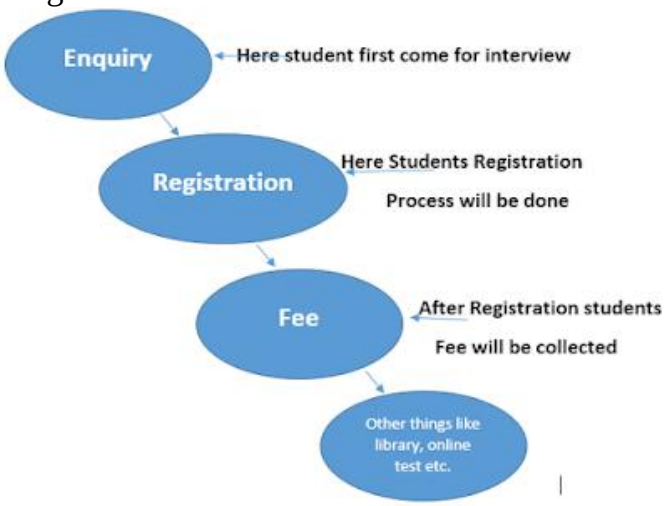
- **When to use Exploratory Testing:-**
  - When you don't have a requirement/testing document/minimal documents.
  - You want to complete your application /software testing in short period of time.
  - You have to test the application in the early stage of SDLC.
  - The testing team has experienced testers
  - Early iteration is required
  - There is a critical application
  - New testers entered into the team
- **Advantages**
  - It doesn't require preparation and documents for testing.
  - Testing time saves due to designing and executing test case or scenarios.
  - Tester can report many issues due to incomplete requirement or missing requirement document.
  - It finds critical defects very quickly than normal testing.
  - Testers use their knowledge, skills and experience to executing more test cases which improves productivity of software.
- **Disadvantages**
  - It is not possible to repeat same test methodology.
  - Once testing is performed it is not reviewed.
  - Keeping track of tests performed is difficult.
  - testing purely depends on the tester skills
  - Defects can be left in this type of testing because nothing is documented

Ad-hoc Testing

- Ad hoc testing is an informal testing technique in which an application is tested without a documented test plan or procedure. Also known as Random Testing or Monkey Testing.
- Ad-hoc testing aim is to break the system i.e. to find defects.
- Ad-hoc testing is doesn't follow formal process like requirement documents, test plan, test cases, etc. it is randomly testing any part of application.
- Only those testers who have good and deep knowledge of the software to conduct this test.
- Ad-hoc testing can be achieved "**Error Guessing technique".** This testing is to find out the potential areas of the software, where errors could be present.
- Ad-hoc testing can be done at any time whether it's beginning, middle or end of the project.
- This testing used when time is very limited and detailed testing is required.



For example, consider an educational software and tester are performing Ad-hoc Testing on it, usually and common modules of education management software are enquiry module, Registration Module, online test, fee module, library module, store module and many more. The flow looks like as shown in the following image.



While performing Ad-hoc Testing, tester not necessary start the testing with first module i.e. Enquiry module, tester may start testing with any module like it may start with fee or it may start with registration. It is totally depends on testers thing which module can cause or have bug. Tester uses error guessing approach to guess the bug in the system. Ad-hoc Testing Does not have any flow. If tester gets some bug then it will be reported to concern developer. Main problem of Ad-hoc Testing is to reproduce bug. It becomes difficult to reproduce the bug as the tester does not have any steps written.

- **Types of ad-hoc testing**

Buddy Testing	One person come from development team and other person come from Testing team and they both assigned to a module. Buddy testing helps the testers develop better test cases and development team can also make changes early. This testing usually happens after unit testing completion.
Pair testing	Two testers are assigned modules, share ideas and work on the same machines to find defects. Both person who are going to test software, One person can execute the tests and another person can take notes on the findings. Pair testing is done only with the testers with different knowledge levels.(Experienced and non-experienced to share their ideas & views)
Monkey Testing	Randomly test the product or application without test cases with a goal to break the system.

- **Advantages of Ad-hoc testing:**
  - It saves the time.

- Does not require any documentation work.
- Useful when we do not have sufficient software specification.
- Easy to start, does not require pre-planning.

➢ **Disadvantages of Ad-hoc testing:**

- More difficult to map defects to test cases because testers can't keep track of STR's (steps to reproduce an error).
- Potential to miss critical bugs - cannot use in final stage testing.
- High chance of redundant testing
- Requires experienced testers who have in-depth knowledge about the software.

Adhoc Testing	Exploratory Testing
Learning software before testing	Learning and testing the software simultaneously
There is no time limit to perform this test	It has a time limit
It can be used for simple functions that may not require much documentation	It can be used for complex functions where little documentation is required
Testers should have prior knowledge of the system	A tester doesn't need to have any prior knowledge of the system. Tester can learn about the system at testing time
It follows a less practical approach to testing	It follows comparatively a more practical approach to testing

Performance Testing

Performance testing is the process of determining how a system responds under a particular workload or task. This testing tests the responsiveness, stability, scalability, reliability, speed and resource usage of your software and infrastructure. It is a type of non-functional testing.

The focus of Performance Testing is validate and verify quality attributes of the system such as

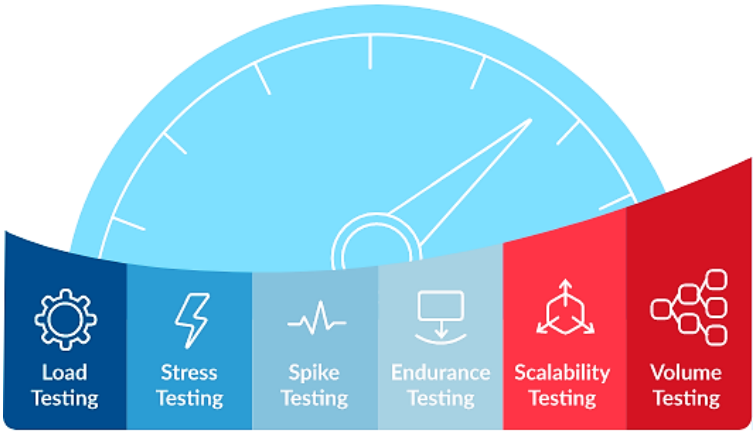
- ***Speed*** **-** Determines whether the application responds quickly
- ***Scalability*** **-** Determines maximum user load the software application can handle.
- ***Stability*** **-** Determines if the application is stable under varying loads

Features or Characteristics of Performance Testing are:

- To generate load on the system to be tested.
- To measure the timing of specific transactions as the load on the system varies.
- To measure average response times.
- To produce graphs or charts of responses over time

The goal of performance testing is not only find the bugs in the system but also eliminate the performance blockages from the system.

The system is tested under a mixture of load conditions and check the time required responding by the system under changeable workloads. It is ensure that application is working under the expected variety of load conditions.



Without doing performance testing may cause the issues like running system slow while simultaneously accessing system by several users, poor usability which likely to increase the bad

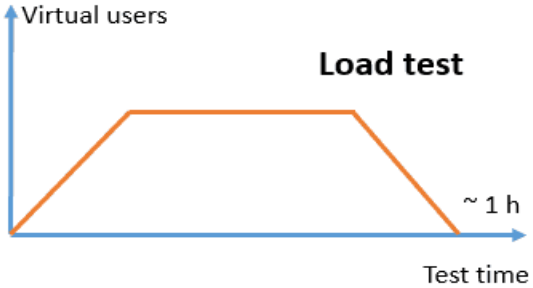
reputation and it affects the expected sales goal directly. Performance testing includes a range of different tests which enable analysis of various aspects of the system.

The basic parameters monitored during performance testing include:

Load Testing:

Load testing is used to study the behavior of the application under specified loads. It also shows how an application will functioning when the majority of its users are logged in. After increasing number of concurrent users, transactions to check the behavior of application under test. Load testing is mainly done to measure response times, resource utilization, and throughput rates. It is all about testing behavior under normal and peak workload conditions.

The application will be slower under heavy load, load testing is to see whether the application can maintain the increased load on the server or will it crash and kill the servers. Load is describing your actual workload. Load testing is usually started as low numbers and slowly increased over a given period of time until it reaches the desired load on the system and then it slope down.



Load testing generally refers to an expected usage of a software program by simulating multiple users accessing the program's services concurrently. This testing is most applicable for multi-user systems, client/server model, such as web servers. However, other types of software systems can be load-tested also. Load testing comes under the Non Functional Testing& it is designed to test the non-functional requirements of a software application.

Goals of Load Testing:

- Response time for each transaction
- Performance of System components under various loads
- Performance of Database components under different loads
- Network delay between the client and the server
- Software design issues
- Server configuration issues like Web server, application server, database server etc.
- Hardware limitation issues like CPU maximization, memory limitations, network blockage, etc.

Example: a word processor can be forced to read or does a spell check on an extremely large document.

A web site with shopping cart capability is required to support 100 concurrent users broken out into following activities:

- 25 Virtual Users log in, browse through items and then log off
- 25 Virtual Users log in, add items to their shopping cart, check out and then log off
- 25 Virtual Users log in, return items previously purchased and then log off
- 25 Virtual Users just log in without any subsequent activity

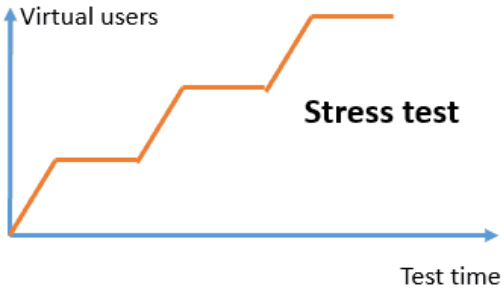
Stress Testing:

Stress Testing is done to calculate the application's actions beyond normal or peak load conditions. It is performed to determine the upper limit of the application capacity and how the application performs when the current load exceeds the expected maximum. The primary focus of performing testing is to identify application bugs that occur in high load conditions. This test determines the maximum load that a given application can support.

It is basically testing the functionality of the application under high loads. It involves testing beyond normal operational capacity to determine breaking points or safe usage limits. It may create unexpected integrity issues in a variety of ways like

- Closing and restarting database access suddenly
- Randomly shutting down network connections
- Severely reducing physical resources such as disk space, memory or network bandwidth

Stress Testing is a type of Non-Functional Testing. Stress testing is Negative testing to check the stability of software when hardware resources are not sufficient where we load the software with large number of concurrent users/processes which cannot be handled by the systems hardware resources like CPU, memory, disk space etc. The purpose behind stress testing is to find out the failure of system and to monitor how the system recovers back gracefully. If the system is able to recover without loss of data and without creating security leaks, it means that it has successfully passed **the stress test.** It is all about growing issues under extreme conditions and resource failures.



Stress testing is to determine the limit, at which the system or software or hardware breaks.

Need for Stress Testing

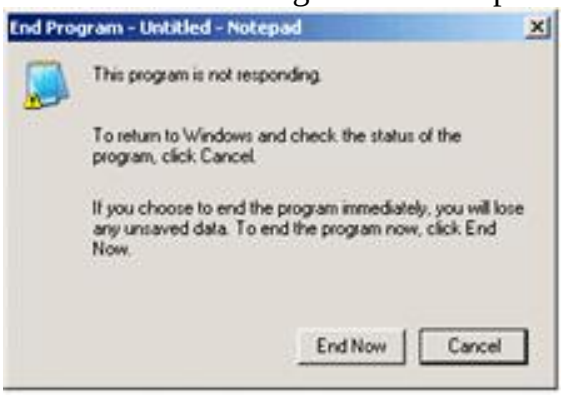
- To check whether the system works under abnormal conditions.
- Displaying proper error message when the system is under stress.
- System failure under extreme conditions could result in huge loss

Consider the following scenarios -

- During festival time, an online shopping site may witness a spike in traffic, or when it announces a sale.
- When a blog is mentioned in a leading newspaper, it experiences a sudden surge in traffic.

It also checks whether system show effective error management under extreme conditions.

Example: The application under testing will be stressed when 5GB data is copied from the website and pasted in notepad. Notepad is under stress and gives 'Not Responded' error message.



Volume testing

Volume testing is non-functional testing which refers to testing a software application with a large

amount of data to be processed to check the efficiency of the application. The main goal of this testing is to evaluate the system performance by increasing the volume of data in the database. Volume testing will to verify the physical and logical limits to a system's capacity.
For example, testing the music site behavior when there is millions of user to download the song.
In volume testing, following things need to be tested

- Test to check if there is any data loss
- Check the system's response time
- Check if the data is stored correctly or not
- Verify if the data is overwritten without any notification
- Check for warning and error messages, whether it comes at all for volume problems
- Check whether high volume data affects the speed of processing

Benefits of Volume Testing

- By identifying load issues, a lot of money can be saved which otherwise will be spent on application maintenance.
- It helps in quicker start for scalability plans
- Early identification of blockage
- It assures your system is now capable of real world usage

The objective of performing the volume testing is to

- Check system performance with increasing volumes of data in the database
- To identify the problem that are likely to occur with large amount of data
- To figure out the point at which the stability of the system degrades
- Volume Testing will help to identify the capacity of the system or application - normal and heavy volume

Soak Testing

Soak testing is a non-functional testing in which, the System under Load (SUL) is tested & verified so that it can continue a huge volume of the load for an extended period of time. It is testing at a system level, to find whether the system will stand up to a very high volume of usage and to see what would happen outside its design expectations. This type of test can identify issues relating to memory allocation, log file handles, and database resource utilization.

A system may behave normally when used for 2 hours, but when the same system is used continuously for 10 hours or more than that then it may fail or behave abnormally/randomly/it may crash. To predict such failure Soak Testing is performed.

The best time to do a soak testing is over the weekends as an application needs to be in a running state for as long as over a day or night. It totally depends on the limitations of the testing situation. Soak tests are one of the most important compliance requirements which need to be very strictly followed by every company.

Example:

- Flipkart is announcing a large discount offering on product. They may expect increase in number of users for 2-3 days. The purpose of soak testing is to create such situations. This helps in verifying if system can carry on too much load for long duration.
- A banking environment large amount of load is expected at the deadline of every month, means on the time of **banking closing days**. So tester always perform the **soak testing** on banking application to test huge levels of load for prolonged amount of time. So in banking application tester put the grand amount of load for extended amount of time **like put load for 72hrs to 120 hrs**, to check how application behaves during this load period.

Regression testing

Regression testing means we develop an application and test it to check whether its functions are working fine. Now, we make changes to it and again we need to test to check whether the functions are still working perfectly in addition to those new changes. i.e. retesting the unchanged parts of the application. "The main aim of regression testing to make sure that changed component is not impacting the unchanged part of the component".

Regression Testing is the one in which test cases are re-executed in order to check whether the previous functionality of the application is working fine and the new changes have not introduced any new bugs. This test can be performed on a new build when there is a major change in the original functionality that too even in a single bug fix. Verifying that the bugs are fixed and the newly added features have not created any problem in the previous working version of the software.

Retesting is done to make sure that bug is fixed and failed functionality is working fine or not, This is kind of verification method followed in testing field for the fixed bugs. After a defect is detected and fixed, the software should be retested to confirm that the original defect has been successfully removed. This is called Confirmation Testing or Re-Testing.

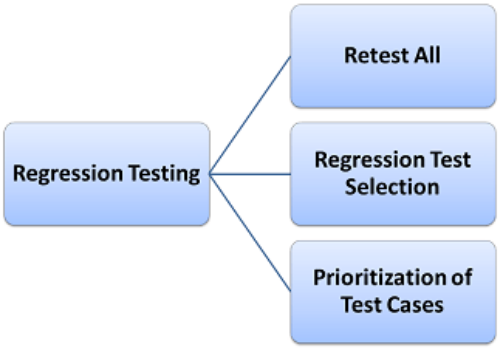
➢ Need of Regression Testing

We do execute regression tests in the following cases:

- When new functionalities are added to the application.
  Example: A Gmail has a login functionality which allows users to do login only with Email. Now the new features look like "to do login using Mobile".
- When there is a Change Requirement.
  Example: Remember password should be removed from the login page which is available earlier
- When there is a Defect Fix.
  Example: Imagine Login button is not working in a login page and a tester reports a bug stating that the login button is broken. Once the bug is fixed by the developers, testers test it to make sure whether the Login button is working as per the expected result. Simultaneously testers test other functionalities which are related to login button
- When there is a Performance Issue Fix.
  Example: Loading the home page takes 5 sec. reducing the load time to 2 sec.
- When there is an Environment change.
  Example: Updating the Database from MySQL to Oracle)

➢ Regression Testing Techniques

Software maintenance is an activity which includes enhancements, error corrections, optimization and deletion of existing features. These modifications may cause the system to work incorrectly. Therefore, Regression Testing becomes necessary. Regression Testing can be carried out using following techniques:



- **Retest All**
  - In which all the tests in the existing test suite should be re-executed. This is very expensive as it requires huge time and resources.
- **Regression Test Selection**
  - Instead of re-executing the entire test suite, it is better to select part of test suite to be run
  - Test cases selected can be categorized as 1) Reusable Test Cases 2) Obsolete Test Cases.
  - Re-usable Test cases can be used in succeeding regression cycles.
  - Obsolete Test Cases can't be used in succeeding cycles.
- **Prioritization of Test Cases**
  - Prioritize the test cases depending on business impact, critical & frequently used functionalities.
  - Selection of test cases based on priority will greatly reduce the regression test suite.

Example: You developed an alarm clock mobile app. This app has just two core functions: Setting the time and activating a sound signal to wake you up. You run functional testing to make sure the app works, get a positive result and rest guaranteed. Later on, you decide that the app may well notify you about important events in your calendar. You add the feature and test it. It works well, but this time you **retest** the wake-up functionality of your app to make sure a new function didn't slow down the existing one.

- **Types of regression testing**
  - **Unit regression** –
    Testing the only one particular fixed unit or feature .Unit regression testing, executed during the unit testing phase, tests the code as a single unit. It has a narrow and focused approach, where complex interactions and dependencies outside the unit of code in question are temporarily blocked.
  - **Regional or Partial regression** –
    Testing the changes and the impact regions is called Regional Regression testing. Partial regression is performed after impact analysis. In this testing process, the new addition of the code as a unit is made to interact with other parts of older existing code, in order to determine that even with code modification, the system functions in silos as desired.
  - **Full or Complete regression** –
    Complete regression testing is often carried out when the code changes for modification or the software updates seep way back into the roots. It is also carried out in case there are multiple changes to the existing code. It gives a comprehensive view of the system as a whole and weeds out any unforeseen problems. A sort of a "final" regression testing is implemented to certify that the build (new lines of code) has not been altered for a period of time. This final version is then deployed to end-users.

- **Advantages of Regression testing:**
  - It helps us to make sure that any changes like bug fixes or any enhancements to the module or application have not impacted the existing tested code.
  - It ensures that the bugs found earlier are NOT creatable.
  - Regression testing can be done by using the automation tools
  - It helps in improving the quality of the product.
- **Disadvantages of Regression testing:**
  - If regression testing is done without using automated tools then it can be very tedious and time consuming because here we execute the same set of test cases again and again.
  - Regression test is required even when a very small change is done in the code because this small modification can bring unexpected issues in the existing functionality.

Regression testing	Smoke testing
Regression testing is used to check defects generated to other modules by making the changes in existing programs.	At the time of developing a software product smoke testing is used.
In regression tested components are tested again to verify the errors.	It permits the software development team to test projects on a regular basis.
Regression testing needs extra manpower because the cost of the project increases.	Smoke testing does not need an extra manpower because it does not affect the cost of project.
Testers conduct the regression testing.	Developer conducts smoke testing just before releasing the product.

Recovery Testing

Recovery Testing verifies software's ability to recover from failures like software/hardware crashes, network failures etc. The purpose of Recovery Testing is to determine whether software operations can be continued after failure or integrity loss. Recovery testing involves reverting back software to the point where integrity was known and reprocessing transactions to the failure point.

A system or software should be recovery tested for failures like:

- Power supply failure

- The external server is unreachable
- Wireless network signal loss
- Physical conditions
- The external device not responding
- The external device is not responding as expected, etc.

Example: When an application is receiving data from the network, unplug the connecting cable.

- After some time, plug the cable back in and analyze the application's ability to continue receiving data from the point at which the network connection was broken.
- Restart the system while a browser has a definite number of sessions open and check whether the browser is able to recover all of them or not

Advantages:

- It helps to make the system more stable and bug-free and improves the quality of the product.
- The system becomes more reliable as bugs are cleared before they go-live and improve the performance of the system.
- Back up is always maintained to recover data in case of any failure.

Disadvantages:

- A trained resource is required to perform this testing. Tester performing the same should have all the data for testing, i.e. data and backup files.
- Recovery testing requires several steps to be performed before the testing and many steps while performing, which makes it a time-consuming process.
- Recovery testing is an expensive process.
- Not all the potential bugs can be found in a few cases.

****

Questions:

1) What is black box testing? Explain various techniques with example
2) Explain Performance testing with its type.
3) What are the benefits of acceptance testing? Difference between alpha and beta testing.
4) Why smoke testing is needed? Explain it.
5) What is Regression Testing? Explain with types
6) Difference between Ad-hoc and Exploratory testing.